

RAMBØLL / DIGITAL PRODUCT ENGINEERING

Ramboll Developer Hub

A developer operating system for Ramboll

Product Vision & Concept Document

Find what exists. Understand how it works. Start new work correctly.
Connect delivery, code, knowledge, and governance in one place.

Prepared by Kristoffer Pedersen
krtip@ramboll.com

2 July 2026

Contents

1 Product Vision	2
2 Structure — Home	2
3 Software Catalog	2
3.1 Catalog contents	2
3.2 Standard catalog item page	3
4 Knowledge Base	3
5 GitHub / Repository Overlay	4
5.1 Repo Quality / Engineering Readiness Score	5
6 Project Management Integrations	5
6.1 Integrations to consider	5
6.2 Useful cross-system views	6
7 Agent-Powered Ideas	6
7.1 Developer Agent	6
7.2 Project Agent	7
7.3 Platform Agent	8
8 Ramboll Design Language Direction	8
8.1 Design principles	8
8.2 Visual language	9
9 Proper Hub Landing Page	9
10 Search Experience	9
11 Golden Paths	10
12 Big Idea Bank	11
12.1 Developer experience	11
12.2 Knowledge	12
12.3 Repo intelligence	12
12.4 Delivery and project	13
12.5 Platform engineering	13
12.6 Agent-powered	14
12.7 Governance	15
13 Suggested MVP	15
13.1 MVP scope	15
13.2 MVP sidebar	16
14 Strong Product Framing	16

1 Product Vision

A Ramboll Developer Hub: one internal place where developers, data engineers, platform teams, architects, DevOps, and product teams can discover systems, understand ownership, follow approved engineering patterns, launch new projects, access documentation, and automate routine work.

*The hub should not be like “another documentation site.” It should be a **developer operating system** for Ramboll.*

- Find what exists.
- Understand how it works.
- Start new work correctly.
- Connect delivery, code, knowledge, and governance in one place.

2 Structure — Home

A clean sidebar organized around **developer intent**, not internal team structure. The home dashboard is the entry point and must give users a reason to return daily.

Area	Content
My work	Assigned issues, pull requests, reviews, incidents, deployments
My services	Repositories, APIs, components, data products, cloud resources
Recommended actions	Outdated docs, failing checks, missing ownership, security alerts
Recently viewed	Repos, services, docs, project pages
Announcements	Platform changes, upcoming migrations, new standards
Quick actions	Create service, request access, open support ticket, ask platform agent

Why this matters

A core pain point for a DPE Engineer is working across **multiple projects simultaneously** while maintaining many older projects. The context-switching overhead here is significant and represents a clear opportunity to streamline day-to-day work through a unified, personalized home view.

3 Software Catalog

This is the **core of the IDP** (Internal Developer Platform).

3.1 Catalog contents

Section	Purpose
Services	Applications, APIs, workers, integrations, packages
Systems	Larger domains or product ecosystems
Components	Libraries, packages, shared modules
APIs	REST, GraphQL, events, internal APIs
Data products	Datasets, pipelines, models, dashboards
Cloud resources	Environments, Kubernetes namespaces, databases, storage
Owners	Teams, maintainers, escalation paths
Lifecycle	Experimental, active, deprecated, sunset
Health	Build status, security, reliability, documentation score

3.2 Standard catalog item page

Every catalog item shares the same page structure, so any engineer can navigate any service the same way.

Tab	Content
Overview	Purpose, owner, criticality, lifecycle, tech stack
Docs	Linked docs, README, ADRs, runbooks
Source	GitHub repo, open PRs, recent commits
Deployments	Environments, versions, release history
APIs	Consumed and exposed APIs
Dependencies	Upstream/downstream systems
Incidents	Past incidents, known risks, support status
Security	Vulnerabilities, secrets, access model
Cost	Cloud cost, ownership, usage trends
Agent	Ask questions, generate docs, summarize risks

4 Knowledge Base

The knowledge base should be more than a wiki: **linkable, searchable, versioned, and connected to code.**

Section	Example content
Getting started	Onboarding, dev setup, accounts, tools
Golden paths	"Create a new API", "Deploy to Azure", "Add monitoring"
Engineering standards	Coding standards, testing, branching, reviews
Architecture	Reference architectures, ADR templates, patterns
Security	Secrets, access, threat modeling, dependency rules
Cloud & infrastructure	Landing zones, Kubernetes, Terraform, environments
Data & AI	Data governance, ML lifecycle, model deployment
APIs & integration	API design, eventing, authentication
DevOps	CI/CD, releases, rollback, observability
Incident response	Runbooks, escalation, postmortems
FAQ	Common questions and troubleshooting

Key feature

Every knowledge article must be linkable from repos, PRs, issues, catalog pages, and project pages.

This repository follows:

- *Ramboll API Golden Path*
- *Ramboll TypeScript Service Standard*
- *Ramboll Azure Deployment Pattern*
- *Ramboll Incident Runbook Template*

5 GitHub / Repository Overlay

A **Ramboll Repo Overlay**: a layer above GitHub (and Azure DevOps) that gives every repository a standardized developer-portal page. Instead of replacing GitHub, the hub *enriches* it.

Area	Content
Repository identity	Name, description, owner, domain, lifecycle
Compliance status	README, CODEOWNERS, license, security policy, docs
Engineering health	Build status, test coverage, open PRs, stale branches
Security	Dependabot, vulnerability status, secret scanning, permissions

(continued)

Area	Content
Knowledge links	Related docs, ADRs, runbooks, onboarding
Project links	Jira/Azure DevOps/Planner epics, roadmap, delivery status
Agent actions	Explain repo, summarize PRs, generate onboarding, suggest fixes

5.1 Repo Quality / Engineering Readiness Score

Score category	Measures
Documentation	README, ownership, runbook, API docs
Maintainability	Dependencies, test coverage, stale code
Delivery	CI health, deployment frequency, PR age
Security	Vulnerabilities, secrets, branch protection
Ownership	CODEOWNERS, team mapping, escalation path
Platform alignment	Uses approved templates, pipelines, infra patterns

Reality check

Projects today are scattered across **GitLab, GitHub, Azure DevOps, and Bitbucket** — fragmented and low on transparency. The hub should unify visibility *without* removing the ability to keep invite-only repositories where a security clearance is genuinely required (critical infrastructure, military, etc.).

6 Project Management Integrations

The platform should connect engineering work to delivery context.

6.1 Integrations to consider

System	Integration ideas
Jira / Azure DevOps	Epics, issues, sprint status, roadmap
GitHub Issues	PR-to-issue linking, backlog visibility
ServiceNow	Incidents, support tickets, change requests
Microsoft Teams	Notifications, support channels, incident rooms

(continued)

System	Integration ideas
Planner / Project	Lightweight task and portfolio views
Confluence / SharePoint	Existing documentation ingestion
Power BI	Engineering metrics dashboards
Azure / AWS / GCP	Cloud resources, cost, environments
Kubernetes	Services, deployments, pods, namespaces
CI/CD	GitHub Actions, Azure Pipelines, deployment history
Security tools	SAST, dependency scanning, secret scanning
Observability	Logs, traces, alerts, SLOs

6.2 Useful cross-system views

View	Purpose
Project page	Epic, repos, services, environments, docs, releases
Team page	Owned services, current work, risks, support load
Delivery timeline	Commits, PRs, deployments, releases, incidents
Dependency map	Shows which teams depend on which systems
Risk dashboard	Security, aging dependencies, missing ownership
Release cockpit	Change requests, approvals, deployments, rollback plan

7 Agent-Powered Ideas

Agent features should not just be chat — they should be **context-aware actions** inside the hub.

7.1 Developer Agent

Question	Example
Repo understanding	"What does this service do?"
Onboarding	"How do I run this locally?"
Architecture	"What depends on this API?"

(continued)

Question	Example
Debugging	"Why did the last deployment fail?"
Standards	"Does this repo follow Ramboll's API standard?"

Action	Description
Generate README	Based on repo structure and catalog metadata
Generate ADR draft	From architecture decision notes
Explain PR	Summarize changes, risks, affected systems
Create onboarding guide	For a repo, team, or platform area
Suggest ownership	Infer owners from commits, CODEOWNERS, teams
Find duplicate services	Detect similar repos or overlapping APIs
Create runbook	Based on alerts, logs, and deployment setup

7.2 Project Agent

Question	Example
Delivery status	"What is blocking Project X?"
Dependency risk	"Which services are on the critical path?"
Sprint summary	"Summarize progress this week."
Release readiness	"Are we ready to release?"
Stakeholder update	"Draft a non-technical status update."

Action	Description
Generate weekly project summary	Based on Jira, PRs, deployments
Identify stale issues	Flags tickets without movement
Link orphaned PRs	Finds PRs not connected to work items
Create release notes	From merged PRs and issues

(continued)

Action	Description
Risk detection	Spots delayed dependencies or failing checks

7.3 Platform Agent

Question	Example
Golden path guidance	"How do I create a new data API?"
Access requests	"How do I get access to environment X?"
Standards	"Which template should I use?"
Migration help	"How do I move this service to the new pipeline?"

Action	Description
Scaffold new service	Uses approved Ramboll templates
Request access	Creates approval flow automatically
Validate repo	Checks against standards
Open platform ticket	With logs/context attached
Recommend golden path	Based on project type

8 Ramboll Design Language Direction

The interface should feel **professional, calm, engineered, and trustworthy** — leaning into clarity, sustainability, engineering confidence, and human-augmented workflows.

8.1 Design principles

Principle	UI expression
Clarity first	Clean layouts, strong hierarchy, minimal clutter
Engineering precision	Structured cards, status labels, metadata tables
Human-centered	Helpful empty states, guided flows, plain language
Sustainable digital	Avoid dashboard overload, prioritize useful signals

(continued)

Principle	UI expression
Trustworthy automation	Agents explain sources, confidence, and actions
Ramboll identity	Use brand colors, typography, spacing, icon style consistently

8.2 Visual language

Component	Style idea
Sidebar	Calm, persistent, grouped by developer intent
Cards	Rounded but professional, with clear metadata
Status chips	Healthy, warning, critical, deprecated, experimental
Service pages	Documentation-like, but with live operational context
Agent panel	Right-side contextual assistant, not a full-screen chatbot
Search	Command-palette style, fast and global
Dependency graph	Clean network visualization with filters
Golden path pages	Step-by-step, with progress and automation buttons

Status chip demonstration: Healthy Warning Critical Deprecated Experimental

9 Proper Hub Landing Page

The landing page should answer: *what can I do here today?*

Ramboll Developer Hub Build, ship, operate, and understand software at Ramboll.
 [Search everything...]
 Quick actions: [Create new service] [Find API] [Request access] [Ask agent]
 My work: - 3 PRs awaiting review - 2 failing builds - 1 deployment pending approval - 4 assigned tickets
 My services: - Customer Data API - Project Insights Portal - Emissions Calculator
 Recommended: - Add CODEOWNERS to repo X - Update deprecated dependency in service Y - Review API documentation for service Z
 Platform updates: - New Azure deployment golden path available - GitHub Actions template v3 released - Security policy update effective 12 July

10 Search Experience

Search should be one of the most important features in the hub, spanning every content type.

Content type	Examples
Repositories	GitHub repos, packages, libraries
Services	APIs, components, systems
Docs	Knowledge articles, ADRs, runbooks
People	Owners, maintainers, experts
Projects	Epics, initiatives, roadmaps
Tickets	Jira, Azure DevOps, ServiceNow
Incidents	Past incidents, postmortems
APIs	Endpoints, schemas, consumers
Cloud resources	Namespaces, environments, databases

Example search result

Customer Data API

Service · Owned by Data Platform · **Production**

Docs: **94%** Security: **Healthy** Last deployed: *2 days ago*

[Open service](#)
[View repo](#)
[Ask agent](#)

11 Golden Paths

Golden paths should be **first-class** objects in the hub, not buried documentation.

Golden path	Description
Create new web app	Repo, CI/CD, hosting, monitoring, docs
Create new API	OpenAPI, auth, deployment, ownership
Create data pipeline	Data source, validation, lineage, monitoring
Create AI assistant	Model selection, data rules, evaluation, governance
Deploy to Azure	Infrastructure, secrets, pipeline, observability
Add observability	Logs, traces, metrics, dashboards
Add security scanning	SAST, dependencies, secret scanning
Create integration	Event/API patterns, authentication, monitoring

(continued)

Golden path	Description
Production readiness	Checklist before go-live
Deprecate service	Communication, migration, sunset plan

Anatomy of a golden path page

Area	Content
When to use	Clear use cases
What you get	Repo, pipeline, infra, docs, monitoring
Prerequisites	Access, approvals, required info
Steps	Manual and automated steps
Templates	Approved starters
Standards	Linked policies and examples
Agent help	“Guide me through this”

12 Big Idea Bank

A backlog of ideas beyond the MVP, grouped by theme.

12.1 Developer experience

Idea	Description
Global command palette	Search and run actions from anywhere
Personalized dashboard	Shows only relevant services, teams, PRs, tickets
Team homepages	Ownership, services, projects, docs, support
“Who owns this?” button	Quickly find responsible team/person
Engineering glossary	Internal terms, acronyms, platform concepts
New joiner mode	Guided onboarding for new developers
Local setup assistant	Repo-specific environment setup help

(continued)

Idea	Description
Dev environment templates	Preconfigured containers or Codespaces
One-click repo bootstrap	Creates repo with standards built in
Self-service access	Request, approve, audit access in one flow

12.2 Knowledge

Idea	Description
Docs quality score	Measures freshness, ownership, completeness
Auto-stale detection	Flags outdated docs
Docs from code	Generate/update docs from repo metadata
ADR library	Searchable architecture decision records
Pattern library	Approved implementation examples
Runbook library	Operational guides linked to services
“Explain this system” pages	Human-readable system summaries
Architecture map	Visual maps of systems and dependencies
Internal Stack Overflow	Q&A with accepted answers
Expert finder	Find people who know a domain or repo

12.3 Repo intelligence

Idea	Description
Repo health score	Docs, security, CI, ownership, freshness
Missing metadata detector	Flags repos without owner/lifecycle/domain
Dependency freshness	Highlights outdated packages
PR risk scoring	Detects large, risky, or sensitive PRs
Review load dashboard	Shows bottlenecks in code review

(continued)

Idea	Description
Branch hygiene	Finds stale branches
Test maturity score	Unit/integration/e2e coverage overview
Security posture	Vulnerabilities, secrets, permissions
License compliance	Detect risky licenses
AI-readiness score	Checks if repo is understandable by agents

12.4 Delivery and project

Idea	Description
Project cockpit	One page for work, repos, docs, releases
Roadmap view	Links initiatives to actual engineering assets
Release readiness	Checks PRs, tests, approvals, incidents
Automated release notes	From PRs, commits, tickets
Dependency risk map	Shows cross-team blockers
Sprint intelligence	Summaries from tickets and PRs
Stakeholder summary generator	Non-technical progress updates
Change approval integration	Link deployments to approvals
Portfolio health	Engineering status across initiatives
Delivery traceability	Requirement → issue → PR → deploy → service

12.5 Platform engineering

Idea	Description
Self-service infrastructure	Provision environments, databases, queues
Approved templates	Standardized service starters
Platform marketplace	Reusable internal capabilities

(continued)

Idea	Description
Environment catalog	Dev, test, staging, prod overview
Cost visibility	Cloud cost by team/service/project
Quota management	Resource limits and usage
Migration center	Track movement to new platforms
Deprecation center	Manage old systems and sunset plans
Incident integration	Connect services to alerts and incidents
SLO dashboard	Reliability targets per service

12.6 Agent-powered

Idea	Description
Ask this repo	Chat with repository context
Ask this service	Chat with service docs, logs, APIs
Ask this project	Chat with tickets, PRs, roadmap
PR summarizer	Explains changes and risks
Incident summarizer	Explains what happened and next steps
Migration assistant	Guides teams through platform changes
Compliance assistant	Checks standards and suggests fixes
Documentation assistant	Drafts missing docs
Architecture assistant	Suggests patterns and flags risks
Support triage agent	Routes platform questions to right owner
Release agent	Prepares release checklist and notes
Onboarding agent	Creates personalized learning path
Security agent	Explains vulnerabilities and remediation
Cost agent	Explains cloud spend anomalies
Dependency agent	Explains impact of changing a service

12.7 Governance

Idea	Description
Ownership registry	Every service has accountable owners
Lifecycle management	Experimental, active, deprecated, retired
Production readiness checklist	Mandatory before go-live
Architecture review workflow	Lightweight review and approval
Security review workflow	Threat model and scanning evidence
Data governance checks	Classification, lineage, access
Audit trail	Who changed what, when, and why
Policy-as-code dashboard	Shows compliance with standards
Exception management	Track approved deviations
Risk register	Technical risks linked to systems

13 Suggested MVP

Start with a version that is valuable quickly.

13.1 MVP scope

Feature	Why
Home dashboard	Gives users a reason to return
Software catalog	Core of the platform
Repo overlay	Makes GitHub more discoverable and governed
Knowledge base	Centralizes standards and guides
Global search	Reduces tool fragmentation
Golden paths	Drives standardization
Project integration	Connects code to delivery
Basic agent	Helps users ask questions across docs/repos

13.2 MVP sidebar

- Home
- Search
- Software Catalog
- Repositories
- Projects
- Knowledge Base
- Golden Paths
- Teams
- Integrations
- Agents
- Platform Support
- Admin

14 Strong Product Framing

The Ramboll Developer Hub is a unified internal platform for discovering software, understanding ownership, following engineering standards, connecting project delivery to code, and using AI-assisted workflows to reduce friction across the software lifecycle.

Or, shorter:

A single hub for Ramboll engineers to find, build, ship, operate, and improve software.

Best next step

Turn this into a one-page concept document with vision, user groups, sidebar IA, MVP, and roadmap.

Kristoffer Pedersen · krt@ramboll.com

Ramboll · Hannemanns Allé 53 · DK-2300 Copenhagen S · Denmark

<https://ramboll.com/energy>

Ramboll Danmark A/S · CVR/DK REG. NO. 35128417

Registered office: Hannemanns Allé 53, 2300 Copenhagen S